

이화여자대학교 졸업프로젝트 2차 보고서

FINNUT

대학(원)생의 생활비 부담 완화를 위한
소비습관 개선 및 장학·복지 정보 추천 어플리케이션

24팀 이루리

송유진 (팀장, AI & BE) 정서진 (FE) 임슬민 (BE)

지도교수: 황의원

2026년 5월 11일

1. 과제 요약

요약 대학(원)생은 재정 관리 경험이 부족하여 비계획적 지출을 하는 경우가 많다. 또한 이들을 위한 다양한 장학금 및 복지 정책이 존재하지만, 관련 정보가 여러 사이트에 분산되어 있어 탐색하고 활용하는 데 어려움이 있다. 본 프로젝트는 소비 분석 및 맞춤형 3분 코칭 기능과 장학·복지 정보 자동 매칭 기능을 통해 이 두 가지 문제를 하나의 앱에서 동시에 해결하는 모바일 어플리케이션 **FINNUT**을 제안한다.

1.1 문제 정의 (Pain Point)

① 목표 사용자 (Target Customer)

2020년 금융위원회 금융교육 실태조사에 따르면, 일반 국민의 68.6%가 자신의 금융지식이 충분하지 않다고 응답하였으며, 58.4%는 자신의 재무관리 수준이 낮다고 인식하고 있는 것으로 나타났다. 또한 92.4%는 금융교육을 받은 경험이 없으며, 현재 고등학교 금융교육은 교과서의 후반부에 배치되어 있어 교육적 중요도가 상대적으로 낮게 인식되고 있으며, 학생들은 충분한 준비 없이 실제 금융생활에 진입하게 되는 구조적 문제를 안고 있다.¹²

FINNUT의 목표 사용자는 이러한 문제를 겪고있는 대학생 및 대학원생이다. 구체적으로는 월 고정 수입이 없거나 아르바이트·장학금 등에 의존하는 불규칙한 소득 구조를 가지며, 처음으로 독립적인 생활비 관리를 경험하는 학생들을 주요 대상으로 한다.

이들은 아직 체계적인 돈 관리 습관이 자리잡지 못한 상태에서 생활비, 등록금, 주거비 등 다양한 지출을 스스로 감당해야 하는 상황에 놓여 있다. 또한 소득 분위나 학점 등의 조건을 충족하면 장학금·청년 복지 정책의 수혜 대상이 될 수 있음에도 불구하고, 관련 정보의 분산으로 인해 이를 인지하지 못하거나 신청 기회를 놓치는 사례가 빈번하게 발생한다.

② 문제점 (Pain Points)

1. 비계획적인 소비습관으로 인한 생활비 부담 증가

대학생 및 대학원생은 충동구매, 불필요한 구독 서비스 유지, 술자리·모임 등 특정 상황에서의 과소비 등 비계획적인 소비 경향을 보이는 경우가 많다. 문제는 자신이 어디에 얼마를 쓰는지 구체적으로 파악하지 못하기 때문에, 이러한 소비가 문제라는 것을 인식하지 못한 채 반복되어

¹ 이충헌. 「10명중 9명 ‘금융교육 못 받아’... 대책은 4개월째 감감무소식」, *ChosunBiz*, 2020년 4월 20일, 금융위원회·한국갤럽 금융교육 실태조사 인용.

² 한진수. (2017). 왜 학교 금융교육이어야 하는가?. 금융교육연구, 2, 25-49.

습관으로 굳어진다는 점이다. 그 결과 저축 여력은 줄어들고 매달 생활비 부담이 누적되는 악순환으로 이어진다.

2. 분산된 장학·복지 정보로 인한 맞춤 지원 탐색의 어려움

장학금, 청년 주거 지원, 청년 적금 등 대학생·대학원생을 위한 다양한 복지 및 지원 정책이 존재하지만, 관련 정보가 한국장학재단, 정부24, 각 지자체 사이트 등 여러 곳에 나뉘어 있어 자신의 조건에 맞는 정책을 빠짐없이 확인하기 어렵다. 결국 모든 사이트를 일일이 찾아보는 것은 현실적으로 쉽지 않기 때문에, 우연히 눈에 띄는 정책만 신청하게 되어 실제로 받을 수 있는 지원을 충분히 활용하지 못하는 상황이 발생한다.

1.2 유사 서비스와의 비교 및 차별성

본 프로젝트는 국내 대표 금융 플랫폼인 토스와뱅크샐러드를 비교 대상으로 선정하였다. 토스는 20대 가입률이 약 95%³에 달할 만큼 대학생들을 포함한 젊은 층에서 가장 널리 사용되는 금융 앱으로, 소비 분석 기능을 제공한다는 점에서 FINNUT과 기능 영역이 겹친다. 뱅크샐러드는 마이데이터 기반 자산 통합 관리를 중심으로 소비 패턴 분석과 개인화 서비스를 제공한다는 점에서 유사하다. 그러나 두 서비스 모두 소비 분석 이후 실질적인 행동 교정으로 이어지지 않으며, 대학생·대학원생을 위한 장학금 및 복지 정보는 제공하지 않는다. 본 과제는 이러한 한계를 기준으로 FINNUT의 차별성을 도출하고자 두 서비스를 비교 대상으로 선정하였다.

비교 대상 ①: 토스

토스는 간편 송금 서비스를 기반으로 출발하여 현재는 마이데이터 API를 활용한 계좌·카드 자동 연동, 카테고리별 소비내역 분석, 향후 예상 소비 그래프, 신용점수 조회, 금융상품(대출·보험·카드) 추천 등 다양한 금융 기능을 하나의 앱에서 제공하고 있다.

비교 대상 ②: 뱅크샐러드

뱅크샐러드는 마이데이터 기반 종합 자산관리 서비스로, 계좌·카드·대출·투자 자산의 통합 조회, 카테고리별 소비내역 분석 및 월별 지출 통계, 카드 혜택 자동 계산, 개인 금융 코칭

³ 비바리퍼블리카. "토스, 누적 가입자 수 3,000만 시대 열다". 토스피드, 2025년 8월 6일.

리포트(금융비서) 제공, 건강검진 데이터 연계 보험 추천까지 금융과 건강 데이터를 결합한 개인화 서비스를 제공하고 있다.

아래는 기존 서비스와의 비교를 요약한 내용이다.

	토스	뱅크샐러드	FINNUT
주요기능	· 마이데이터 API를 활용한 계좌·카드 자동 연동 및 소비데이터 수집 · 카테고리별 소비내역 분석 및 월별 지출 통계 · 향후 예상 소비 그래프 제공 (소비 리포트) · 신용점수 무료 조회 및 변동 알림 (내 신용) · 대출·보험·카드 추천 (금융상품) · 계좌 상관없이 간편 송금 (송금)	· 마이데이터 API를 활용한 계좌·카드 자동 연동 및 소비데이터 수집 · 카테고리별 소비내역 분석 및 월별 지출 통계 · 카드 혜택 자동 계산 및 최적 카드 추천 · 개인 금융 코칭 리포트 제공 (금융비서) · 건강검진 데이터 연계 맞춤형 보험 추천 · 대출·카드·보험 등 금융상품 추천	· 카드·페이 푸시 알림을 활용한 소비데이터 수집 · 카테고리별 소비내역 분석 및 월별 지출 통계 · 충동구매 지표(야간 소비·24시간 내 반복 결제·소액 다건) 및 소비 급증 지표(최근 7일 vs 이전 30일 평균 비교) 기반 금융건강지수(FHI) 산출 · AI 기반 맞춤형 3분 코칭 카드 생성 및 행동 미션 추천 · 사용자 정보 기반 장학·복지 정책 자동 매칭 및 마감일 캘린더 등록
한계점	· 카테고리별 소비 내역과 월별 지출 통계는 제공하지만, 충동구매·야간 소비 등 개인의 소비 특성을 진단하거나 개선 방향을 제안하는 기능은 없음 · 소비 내역을 하나의 점수나 지표로 통합하여 재정 상태를 한눈에 파악할 수 있는 기능 없음 · 장학금·청년 복지 정책 관련 콘텐츠를 일부 게시글 형태로 제공하나, 사용자 조건에 맞는 정책을 자동으로 찾아주거나 마감일을 관리해주는 기능은 없음	· 카테고리별 소비 내역과 월별 지출 통계는 제공하지만, 충동구매·야간 소비 등 개인의 소비 특성을 진단하거나 개선 방향을 제안하는 기능은 없음 · 금융비서 리포트를 통해 소비 변화를 요약해주지만, 구체적인 솔루션이나 실천 미션으로 이어지지 않음 · 장학금·청년 복지 정책 관련 콘텐츠를 일부 게시글 형태로 제공하나, 사용자 조건에 맞는 정책을 자동으로 찾아주거나 마감일을 관리해주는 기능은 없음	· 마이데이터 연동을 지원하지 않아 카드·페이 푸시 알림을 통해 소비 데이터를 수집하므로, 자동으로 계좌·카드를 연동하는 토스·뱅크샐러드 대비 수집 가능한 데이터 범위가 제한됨 · 소비 데이터가 충분히 축적되기 전까지는 소비 급증 감지 등 일부 지표 계산이 제한됨 · 대출·보험·카드 등 금융상품 추천 기능 미제공

* 색상 범례

■ 파란색 배경: 토스·뱅크샐러드와 공통으로 제공하는 기능 ■ 주황색 배경: FINNUT만의 차별화 기능 ■ 회색 배경: 비교 대상 서비스가 공통으로 가진 한계

토스와뱅크샐러드는 마이데이터 연동을 통한 계좌·카드 자동 연동, 카테고리별 소비내역 분석 및 월별 지출 통계 제공이라는 공통적인 강점을 가진다. FINNUT 역시 카테고리별 소비내역 분석 및 월별 지출 통계 기능을 동일하게 제공하며, 이를 기반으로 기존 서비스가 해결하지 못한 두 가지 한계를 극복한다.

첫째, 기존 서비스는 소비 데이터를 보여주는 것에 그치지만 FINNUT은 충동구매·야간 소비·소비 급증 등 개인의 소비 특성을 분석하고, 이를 금융건강지수(FHI)라는 하나의 지표로 통합하여 재정 상태를 한눈에 파악할 수 있도록 한다. 나아가 AI 기반 3분 코칭 카드와 행동 미션을 통해 분석 결과가 실질적인 소비 습관 개선으로 이어지도록 유도한다.

둘째, 기존 서비스들은 직장인·자산가 등 폭넓은 연령층을 대상으로 하기 때문에 대학생·대학원생에게 특화된 장학금·복지 정책 정보는 다루지 않는다. 반면 FINNUT은 대학생·대학원생을 주요 타겟으로 하는 서비스인 만큼, 이들이 수혜 대상이 될 수 있는 장학금·청년 복지 정책 자동 매칭 기능을 핵심 기능으로 제공한다. 사용자가 여러 사이트를 직접 방문하지 않아도 자신의 조건에 맞는 지원 정책을 한눈에 확인하고 마감일을 놓치지 않도록 지원한다는 점에서 기존 서비스와 명확히 차별화된다.

다만 현재 FINNUT은 초기 단계로, 마이데이터 연동을 지원하지 않아 카드·페이 푸시 알림을 통해 소비 데이터를 수집하므로 기존 서비스 대비 수집 가능한 데이터 범위가 제한된다는 한계가 존재한다. 향후 서비스 확장 시 마이데이터 API 연동을 도입하여 기존 서비스와 동일한 수준의 자동화된 데이터 수집 환경을 구축할 수 있을 것으로 기대된다.

1.3 제안 내용

본 프로젝트는 대학(원)생의 생활비 부담을 유발하는 두 가지 핵심 문제, 비계획적인 소비습관과 분산된 장학·복지 정보 접근성 문제를 동시에 해결하기 위한 어플리케이션 FINNUT을 제안한다. FINNUT은 이러한 다음과 같은 세 가지 핵심 기능을 중심으로 서비스를 설계하였다.

① 소비 분석 기반 금융건강지수(FHI) 산출

사용자의 카드·페이 거래 알림 데이터를 수집·정규화한 뒤 소비 카테고리를 자동으로 분류하고, 충동구매 빈도(24시간 내 반복 결제·소액 다건), 야간 소비 비율(21시~02시 결제), 소비 급증 여부(최근 7일 평균이 이전 30일 평균 대비 50% 이상 증가) 등을 기준으로 개인의 소비 특성을 진단하여 금융건강지수(FHI)를 산출한다. 단순히 지출 내역을 보여주는 데 그치는 기존 서비스와 달리, 소비 상태를 하나의 정량적 지표로 통합하여 사용자가 자신의 재정 상태를 객관적으로 인식하고 스스로 개선 동기를 가질 수 있도록 돕는다.

② 행동 유도형 맞춤 3분 소비 코칭

기존 금융 서비스는 소비 데이터를 분석하여 보여주지만, 그 결과가 문제 인식과 구체적인 실천으로 이어지지 않는다는 한계가 있다. FINNUT은 이 지점에서 차별화된다. 산출한 FHI와 소비 패턴을 바탕으로 AI 맞춤형 3분 코칭 카드를 생성하여 사용자의 소비 문제를 짚어주고, 오늘 당장 3분 안에 실천할 수 있는 행동 미션을 함께 제안함으로써 분석 결과가 실제 습관 변화로 이어지도록 설계되었다.

③ 장학·복지 정보 자동 매칭 및 일정 관리

한국장학재단 API 및 ~~복지로 크롤링을~~ ~~온통청년 open API~~ 를 통해 수집한 장학금·청년 복지 정책 데이터를, 회원가입 시 입력받은 사용자 정보(~~학교, 학과, 소득분위, 학점 등~~)(~~학적상태, 지역, 나이, 소득분위 등~~)를 기준으로 필터링하여 개인에게 맞는 정책만을 추려 제시한다. 또한 추천된 정책의 신청 마감일을 캘린더에 자동 등록하여 신청 기간을 놓치지 않도록 지원한다. 사용자가 여러 사이트를 직접 돌아다니며 정보를 찾아야 했던 기존 방식에서 벗어나, 나에게 해당되는 정책을 앱이 먼저 찾아주는 방식으로의 전환이라는 점에서 차별성을 가진다.

1.4 기대 효과 및 의의

본 프로젝트는 다음과 같은 효과를 기대할 수 있다.

① 생활비 부담 완화 및 계획적인 소비 습관 형성

코로나 이후 SNS 기반 소비행태 연구에 따르면, 청년층은 과시 소비·동조 소비 등 사회적 요인에 의해 충동구매를 하는 경향이 있으며 이러한 소비는 비계획적 지출로 이어질 가능성이 높은 것으로 나타났다.³ 아르바이트·장학금 등 불규칙한 수입에 의존하며 처음으로 독립적인 지출을

감당해야 하는 대학생·대학원생에게 이러한 소비 경향은 생활비 부담으로 직결된다. FINNUT의 소비 분석 및 코칭 기능은 이러한 충동적·비계획적 소비를 스스로 인식하고 교정할 수 있도록 도와 생활비 부담 완화에 기여한다. 나아가 소비를 의식적으로 관리하는 습관을 대학(원)생 때부터 형성한다면, 이후 본격적인 경제 활동을 시작했을 때 보다 건강한 재무 관리의 토대가 될 수 있을 것으로 기대된다.

② 장학·복지 혜택 수혜 가능성 증가

장학금 및 청년 지원 정책은 충분히 존재하지만, 정보가 여러 사이트에 분산되어 있어 자신에게 맞는 정책을 빠짐없이 찾아보는 것이 현실적으로 어렵다. FINNUT은 사용자 조건에 맞는 정책을 자동으로 추려 한 화면에서 확인할 수 있도록 함으로써 정보 탐색에 드는 시간과 노력을 줄인다. 그 결과 지원 자격이 있음에도 몰라서, 혹은 기한을 놓쳐서 신청하지 못하는 상황을 줄이고 실질적인 수혜 가능성을 높인다.

③ 기존 금융 서비스의 한계를 보완하는 새로운 패러다임 제시

기존 금융 서비스가 "얼마를 썼는가"를 보여주는 데 집중했다면, FINNUT은 "어떻게 쓰고 있는가"를 진단하고 "어떻게 바꿀 수 있는가"까지 제안한다. 데이터 확인에서 끝나는 것이 아니라 소비 진단 → 맞춤 코칭 → 장학·복지 추천으로 이어지는 흐름을 하나의 앱 안에 담았다는 점, 전 연령층·경제활동 인구를 대상으로 하는 기존 서비스들이 메인으로 다루지 않았던 장학·복지 정보와 대학생·대학원생 특화 소비 진단을 핵심 기능으로 제공한다는 점에서 기존 서비스와는 근본적으로 다른 방향성을 가진다.

1.5 주요 기능 리스트

FINNUT은 Android 기반 모바일 앱으로 제공되며, 다음과 같은 핵심 기능들로 구성된다.

① 소비 진단 기능

- 거래 알림 텍스트 수집 및 정규화
- 소비 카테고리 자동 분류
- 주간·월간 소비 패턴 분석
- 금융건강지수(FHI) 산출

② 맞춤형 소비 코칭 기능

- 소비 상태 요약 제공
- AI 기반 3분 코칭 카드 생성
- 즉시 실행 가능한 행동 미션 추천

③ 장학·복지 정보 추천 기능

- 공공 API 기반 정책 데이터 수집
- 사용자 조건 기반 맞춤형 추천
- 신청 마감일 캘린더 자동 등록

④ UI / 서비스 기능

- 로그인 및 사용자 프로필 관리
- 홈 대시보드 (FHI 및 소비 요약)
- 코칭 카드 화면
- 장학금 추천 화면
- 북마크 기능
- 캘린더 기능

1 이충현. 「10명중 9명 '금융교육 못 받아'... 대책은 4개월째 감감무소식」, ChosunBiz, 2020년 4월 20일.

2 한진수. (2017). 왜 학교 금융교육이어야 하는가?. 금융교육연구, 2, 25-49.

3 최우리, 윤한성, 김도일. "20대들의 SNS 충동구매 이대로 괜찮은가?". KBM Journal 7.3 (2023): 59-76.

4 비바리퍼블리카. "토스, 누적 가입자 수 3,000만 시대 열다". 토스피드, 2025년 8월 6일.

2. 과제 설계

2.1 요구사항 정의

FINNUT은 대학(원)생의 생활비 부담 완화를 목표로, 소비 진단·맞춤형 코칭·장학 및 복지 정보 추천이라는 세 가지 핵심 기능을 제공한다. 이를 구현하기 위한 요구사항은 기능 요구사항(Functional Requirements)과 비기능 요구사항(Non-Functional Requirements)으로 구분하여 정의한다.

2.1.1 기능 요구사항 (Functional Requirements)

기능 요구사항은 시스템이 반드시 수행해야 하는 동작을 기술한다. FINNUT의 핵심 기능인 소비 진단 엔진, 맞춤형 소비 코칭, 장학·복지 자동 매칭, 모바일 앱 UI의 네 부분으로 분류하여 정의한다.

구분	요구사항 ID	기능 요구사항	우선순위	관련 모듈
소비 진단 엔진	FR-01	시스템은 사용자의 거래 알림 텍스트를 수신하여 가맹점명, 금액, 일시 등의 정형 데이터로 정규화하여야 한다.	High	텍스트 정규화
	FR-02	시스템은 정규화된 거래 데이터를 기반으로 식비, 교통, 쇼핑, 구독 등 소비 카테고리를 자동으로 분류하여야 한다.	High	카테고리 분류
	FR-03	시스템은 카테고리별 지출 패턴을 집계하여 주간·월간 소비 동향을 분석하여야 한다.	High	지출 패턴 집계
	FR-04	시스템은 Rule 기반 방식으로 금융건강지수(FHI)를 산출하고, ML 기반 방식으로 예측한다.	High	FHI 산출
	FR-05	시스템은 충동구매 빈도·소비 급증 여부 등의 행동 지표를 감지하여 FHI 산출에 반영하여야 한다.	Medium	FHI 산출
맞춤형 소비 코칭	FR-06	시스템은 산출된 FHI와 소비 패턴을 바탕으로 사용자의 재정 상태를 요약하여야 한다.	High	상태 해석
	FR-07	시스템은 재정 상태 요약을 입력으로 AI 기반 맞춤형 3분 코칭 카드를 생성하여야 한다.	High	코칭 카드 생성

구분	요구사항 ID	기능 요구사항	우선순위	관련 모듈
	FR-08	시스템은 사용자가 즉시 실천 가능한 소비 절감 행동 미션을 코칭 카드와 함께 추천하여야 한다.	Medium	행동 미션 추천
장학·복지 매칭	FR-09	시스템은 정책 정보 수집 API를 활용하여 장학금 및 복지 제도 데이터를 수집하고 데이터베이스에 저장하여야 한다.	High	정책 데이터 수집
	FR-10	시스템은 사용자의 학교, 학과, 소득 분위, 학점 등의 조건을 기반으로 적합한 장학·복지 정보를 필터링하여 추천하여야 한다.	High	조건 기반 추천
	FR-11	시스템은 추천된 장학·복지 정보의 지원 마감일을 캘린더 이벤트로 자동 등록하여 사용자에게 일정을 알려야 한다.	Medium	캘린더 등록
UI / 앱	FR-12	시스템은 Android 기반 모바일 앱으로 로그인, 홈, 소비 진단, 코칭 카드, 장학 매칭, 북마크, 캘린더 화면을 제공하여야 한다.	High	Android / FE
	FR-13	시스템은 Firebase를 통해 프론트엔드와 백엔드를 연동하고, 사용자 데이터를 실시간으로 동기화하여야 한다.	High	Firebase 연동

* 우선순위 기준: High – 핵심 기능으로 MVP에 포함, Medium – 사용성 향상에 기여하며 최종 스프린트 내 구현 목표

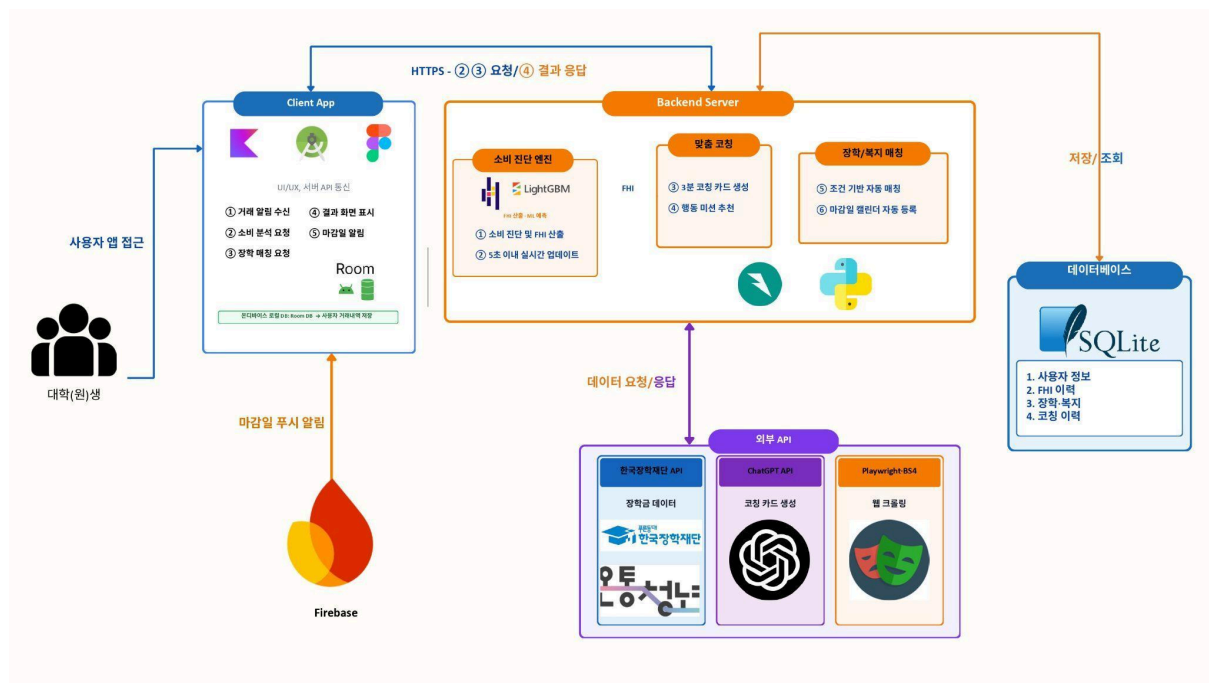
2.1.2 비기능 요구사항 (Non-Functional Requirements)

비기능 요구사항은 시스템의 품질 속성을 정의한다. FINNUT은 금융 데이터를 다루는 특성상 성능과 보안이 특히 중요하며, 장학 정보 연동 구조의 확장성과 AI 모듈 교체를 고려한 유지보수성도 요구사항으로 포함한다.

요구사항 ID	구분	비기능 요구사항	우선순위
NFR-01	성능	거래 알림 수신 후 카테고리 분류 및 FHI 업데이트는 5초 이내에 완료되어야 한다.	High

요구사항 ID	구분	비기능 요구사항	우선순위
NFR-02	보안	사용자의 금융 거래 데이터 및 개인정보는 암호화하여 저장·전송하여야 하며, 인증된 사용자만 접근할 수 있어야 한다.	High
NFR-03	사용성	앱의 주요 기능(코칭 카드 확인, 장학금 추천 조회)은 3단계 이내의 화면 전환으로 접근 가능하여야 한다.	Medium
NFR-04	확장성	장학·복지 데이터 수집 모듈은 새로운 정책 API가 추가될 경우 기존 코드 변경 없이 연동 가능한 구조로 설계되어야 한다.	Medium
NFR-05	유지보수성	Rule 기반 FHI 산출 로직과 ML 기반 예측 모델은 독립적인 모듈로 분리되어야 하며, 모델 교체 시 서비스 중단 없이 적용 가능하여야 한다.	Medium

2.2 전체 시스템 구성 (전체 수정)



[전체 SW구조도]

FINNUT은 **Client App, Backend Server, Firebase, 외부 API, 데이터베이스**의 다섯 구성 요소로 이루어진다.

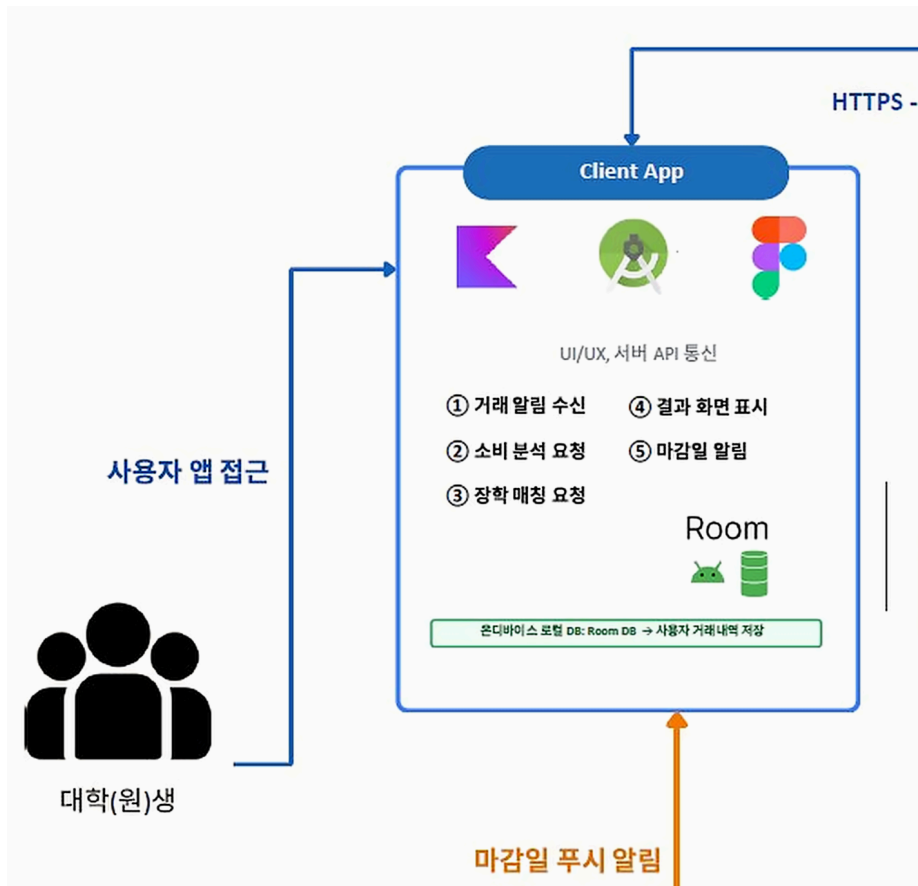
Client App은 Kotlin 기반 Android 앱으로, 사용자의 거래 알림을 수신하여 온디바이스 Room DB에 저장하고, HTTPS를 통해 Backend Server에 소비 분석 및 장학 매칭을 요청한다. Firebase는 장학·복지 신청 마감일 발생 시 FCM 푸시 알림을 Client App으로 전송한다.

Backend Server는 FastAPI 기반 Python Web App으로, 핵심 비즈니스 로직을 세 모듈로 분리하여 처리한다. 소비 진단 엔진은 거래 내역을 분석하여 FHI를 산출하고, 맞춤 코칭 모듈은 FHI를 바탕으로 3분 코칭 카드와 행동 미션을 생성한다. 장학·복지 매칭 모듈은 회원가입 시 입력받은 사용자 조건에 맞게 정책을 자동 매칭하고 마감일을 캘린더에 등록한다. Backend Server는 외부 API로부터 장학·복지 데이터를 수집하며, 해당 데이터는 데이터베이스에 저장된다.

외부 API는 한국장학재단 API, 온통청년 API(장학금 데이터), ChatGPT API(코칭 카드 생성), Playwright-BS4(KOSAF 공지 페이지 동적 크롤링)로 구성된다. 데이터베이스는 SQLite 기반으로 사용자 정보·FHI 이력·장학·복지 데이터·코칭 이력을 저장한다.

각 구성 요소의 상세 설계 및 역할은 아래에서 순서대로 기술한다.

① Client App



Client App은 Kotlin 기반 Android 앱으로, UI/UX 렌더링과 Backend Server와의 API 통신을 담당한다. 사용자와 시스템 간의 접점으로서 다음의 다섯 가지 역할을 수행한다.

거래 알림 수신 사용자 단말기의 카드·페이앱 푸시 알림을 감지하여 수집한 소비 데이터를 온디바이스 Room DB에 로컬 저장한다.

소비 분석 요청 Room DB에 저장된 거래 데이터를 HTTPS를 통해 Backend Server로 전송하여 FHI 산출 및 코칭 카드 생성을 요청한다.

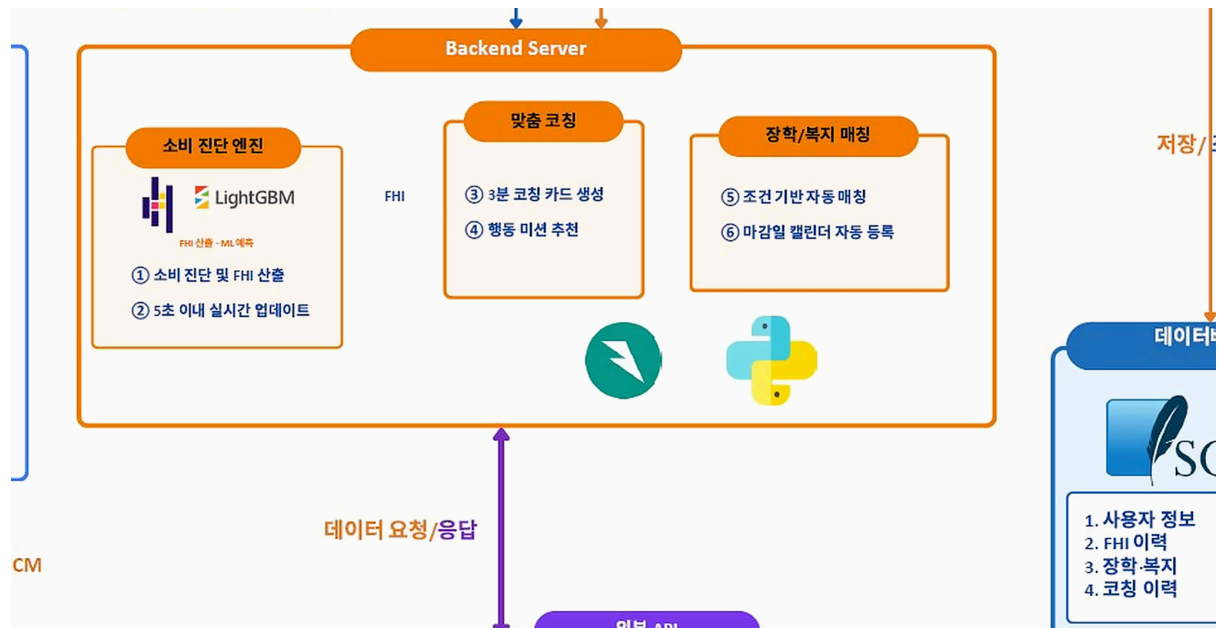
장학 매칭 요청 사용자 조건 정보를 HTTPS를 통해 Backend Server로 전달하여 맞춤 장학·복지 정보 매칭을 요청한다.

결과 화면 표시 Backend Server로부터 수신한 FHI, 코칭 카드, 장학·복지 정보 추천 결과를 화면에 렌더링한다.

마감일 알림 Firebase FCM을 통해 수신한 장학·복지 신청 마감일 알림을 사용자에게 표시한다.

개인 거래 내역은 외부 서버로 전송하지 않고 온디바이스 Room DB에 로컬 저장하여 프라이버시를 보호한다.

② Backend Server



Backend Server는 FastAPI 기반 Python Web App으로, Client App으로부터 HTTPS를 통해 소비 분석 요청과 장학 매칭 요청을 수신하고 처리 결과를 응답한다. 핵심 비즈니스 로직은 소비 진단 엔진, 맞춤 코칭, 장학·복지 매칭의 세 모듈로 구성되며, 각 모듈은 독립적인 API 엔드포인트를 통해 순차적으로 연동된다.

소비 진단 엔진은 Client App으로부터 수신한 거래 데이터를 기반으로 소비를 진단하고 FHI를 산출한다. Rule 기반 FHI 산출과 LightGBM ML 모델을 통한 FHI 예측을 병행하며, 거래 알림 수신 후 5초 이내 실시간 업데이트를 목표로 한다. 산출된 FHI는 데이터베이스에 저장되며 맞춤 코칭 모듈로 전달된다.

맞춤 코칭 모듈은 소비 진단 엔진으로부터 전달받은 FHI와 소비 패턴을 바탕으로 ChatGPT API를 호출하여 3분 코칭 카드를 생성하고, 즉시 실천 가능한 행동 미션을 함께 추천한다. 생성된 코칭 이력은 데이터베이스에 저장된다.

장학·복지 매칭 모듈은 Client App으로부터 전달받은 사용자 조건을 기반으로 외부 API에서 수집한 장학·복지 데이터를 필터링하여 맞춤 정책을 자동 매칭하고, 신청 마감일을 캘린더에 자동 등록한다. 매칭 결과는 데이터베이스에 저장된다.

③ 기타 구성 요소

Firebase는 장학·복지 신청 마감일 발생 시 FCM 푸시 알림을 Client App으로 전송한다. **외부 API**는 한국장학재단 API/온통청년API와 Playwright·BS4 크롤링을 통해 장학·복지 데이터를 수집하고, ChatGPT API를 통해 맞춤형 코칭 카드를 생성한다. **데이터베이스**는 SQLite 기반으로 사용자 정보·FHI 이력·장학·복지 데이터·코칭 이력을 저장·조회한다.

2.3 주요 엔진 설계 및 기능 설계

FINNUT의 핵심 기능은 소비 진단 엔진, 맞춤형 코칭 카드 생성 엔진, 장학·복지 자동 매칭 엔진의 세 모듈로 구성된다. 각 모듈은 독립적인 API 엔드포인트와 처리 로직을 갖추고 있으며, 서로 데이터를 연계하여 하나의 파이프라인으로 동작한다.

2.3.1 소비 진단 엔진 설계

소비 진단 엔진은 거래 알림 텍스트 수신 → 정규화 → 카테고리 분류 → 지출 패턴 집계 → FHI 산출의 5단계 파이프라인으로 구성된다. 각 단계는 독립적인 모듈로 분리되어 유지보수와 기능 교체가 용이하다.

(1) 거래 알림 텍스트 정규화 — `parser.py`

Android의 `NotificationListenerService`를 통해 카드·페이앱의 푸시 알림 텍스트를 수신한다. `_detect_source()`가 텍스트 내 키워드를 감지하여 소스를 자동 판별하고, 소스별 전용 파싱 함수를 적용한 뒤 `_normalize_tx()`를 통해 아래의 공통 스키마로 정규화한다.

필드	설명
<code>datetime</code>	결제 일시
<code>amount</code>	결제 금액 (int, 원)
<code>merchant</code>	가맹점명
<code>category</code>	소비 카테고리
<code>source</code>	알림 소스 (kakaopay / shinhan / kb / samsung / unknown)
<code>payment_method</code>	결제 수단 (wallet / card / unknown)
<code>raw_text</code>	원본 알림 텍스트

공통 스키마는 소비 진단 파이프라인의 후속 단계에서 필요한 정보를 기준으로 설계되었다.

- **datetime** : 야간 소비 감지(21시~02시), 24시간 내 동일 카테고리 2회 이상 반복 결제 감지 등 시간 기반 충동구매 지표 산출에 활용되며, 7일·30일 기준 윈도우 필터링을 통한 지출 패턴 집계 (`day_count_7d`, `day_count_30d`) 산출에도 사용된다.
- **amount** : FHI 산출의 핵심 입력값으로, 소액 다건·소비 급증 지표 계산에 활용된다.
- **merchant** : 카테고리 자동 분류(`category_rules.py`)의 입력값으로, 가맹점명 기반 키워드 매칭에 사용된다.
- **category** : 카테고리별 지출 비율 피쳐(`cat_spend_ratio*_30d`) 산출에 활용된다.
- **source** : 소셜별 파싱 함수 적용 결과를 추적하여 파싱 오류 디버깅 및 미지원 소스 확장에 활용된다.
- **payment_method** : 카드·간편결제 등 결제 수단별 소비 분석의 확장을 고려하여 포함되었다.
- **raw_text** : 파싱 실패 또는 오분류 발생 시 원본 텍스트를 보존하여 디버깅 및 재파싱에 활용된다.

(2) 소비 카테고리 자동 분류 — `category_rules.py`

정규화된 가맹점명에 소문자 변환·지점 접미사 제거(`_norm()`)를 적용한 후 키워드 rule과 매칭하여 카테고리를 분류한다. 매칭 실패 시 'other'로 처리된다.

카테고리 분류 체계는 한국은행 가계금융복지조사의 소비지출 분류 항목을 참고하되, 대학(원)생의 주요 소비처를 고려하여 아래의 9개 카테고리로 재정의하였다.

카테고리 코드	레이블	대표 키워드
shopping_pos	오프라인 쇼핑	올리브영, 무신사, 네일, 헤어, 화장품
shopping_net	온라인 쇼핑	쿠팡, 온라인 장바구니
grocery_pos	편의점·마트	GS25, CU, 세븐일레븐, 이마트24

카테고리 코드	레이블	대표 키워드
other	식비·기타	맥도날드, 버거킹, 김밥, 식당, 커피
gas_transport	교통	지하철, 버스, 택시
entertainment	문화·여가	영화, 전시
home	생활용품	다이소, 생활용품
misc_net	구독	구독
kids_pets	키즈·반려동물	반려동물, 육아

(3) 지출 패턴 집계 — feature_builder.py

build_features_from_transactions()은 정규화된 거래 데이터로부터 20개 피처를 추출한다. 7일·30일 기준 수치 피처 11개와 카테고리별 지출 비율 피처 9개로 구성되며, 이는 **충동구매·소비 급증 지표 산출**과 **ML 기반 FHI 예측**에 활용된다.

구분	피처	활용 목적
7일 수치 (5개)	spend_sum_7d, spend_mean_7d, spend_std_7d, spend_max_7d, day_count_7d	소비 급증 지표 산출 시 "최근 7일 평균" 계산 및 LightGBM 모델 입력값
30일 수치 (6개)	spend_sum_30d, spend_mean_30d, spend_std_30d, spend_max_30d, day_count_30d, unique_category_count_30d	소비 급증 지표 산출 시 "이전 30일 평균" 기준값 계산 및 LightGBM 모델 입력 피처로 활용
카테고리 비율 (9개)	entertainment, gas_transport, grocery_pos, home, kids_pets, misc_net, other, shopping_pos, shopping_net 각 30일 지출 비율	충동구매 지표 산출 시 동일 카테고리 반복 결제 감지 및 LightGBM 모델 입력 피처로 활용

특히 **day_count_7d**와 **day_count_30d**는 소비 빈도를 측정하여 충동구매의 반복 결제 규칙 검증에 활용되며, **unique_category_count_30d**는 소비 다양성을 나타내어 집중적 소비 패턴을 감지하는 데 사용된다. 모든 피처는 Rule 기반 FHI 계산 과정에서 중간값으로 활용되거나, ML 기반 FHI 예측 모델의 직접 입력값으로 사용된다.

(4) FHI 산출 — fhi_calculator.py

추출된 피처를 바탕으로 충동구매 지표(impulsive_score)와 소비 급증 지표(spike_score)를 산출하고, Rule 기반 그리고 ML 기반으로 FHI를 계산한다.

충동구매 지표 산출 — impulsive_detector.py

$$\text{impulsive_score} = 0.4 \times \text{freq_score} + 0.3 \times \text{night_score} + 0.3 \times \text{small_score}$$

3가지 규칙을 가중합하여 impulsive_score(0~1)를 산출한다. 가중치는 충동구매 행동의 위험도를 고려하여 설정되었다.

규칙	조건	가중치	설정 근거
반복 결제 (freq_score)	24시간 내 동일 카테고리 2회 이상 결제	0.4	⁴ 선행 구매가 이후 추가 구매를 유발하는 'shopping momentum' 연구에 근거해, 단기 반복 소비를 가장 직접적인 충동소비 신호로 판단
야간 소비 (night_score)	21시~02시 결제	0.3	⁵ 수면 부족이 구매 후회·반품 가능성과 관련된다는 온라인 쇼핑 연구를 근거로 보조 지표로 설정

⁴ Gilbride, T. J., Inman, J. J., & Stilley, K. M. (2015). *The Role of Within-Trip Dynamics in Unplanned Versus Planned Purchase Behavior*. *Journal of Marketing*, 79(3), 57–73.

→ 쇼핑 과정에서 계획 외 구매가 이후 추가 계획 외 구매 가능성을 높이는 'shopping momentum' 현상을 제시함.

⁵ Gong, S., Zhang, Y., & Gao, Y. (2023). *Sleep deprivation and post-purchase regret: Evidence from a large-scale individual-level data*. *Acta Psychologica Sinica*, 55(2), 286–300.

→ 중국 대형 온라인 쇼핑몰의 개인 단위 거래 데이터 1,625,472건을 분석한 연구로, 수면 부족 상태가 구매 후회 및 반품 선택 가능성과 관련됨을 제시함.

규칙	조건	가중치	설정 근거
소액 다건 (small_score)	1만원 이하 3회 이상	0.3	⁶ 작은 금액의 반복적 지출로 인한 누적 손실

소비 급증 지표 산출 — spending_spike.py

feature_builder.py에서 추출한 spend_mean_7d와 spend_mean_30d를 활용하여 spike_score를 산출한다. 데이터가 10건 미만이거나 이전 30일 데이터가 없는 경우 0으로 처리한다.

$$spike_score = (\text{최근 7일 평균} - \text{이전 30일 평균}) / \text{이전 30일 평균}$$

이때 spike_score $\geq$ 0.3 (30% 증가) 시 과소비로 분류하여 절약을 유도하는 코칭 카드를 생성한다. FHI 계산 시에는 spike_score가 1.0 이하인 경우 그대로 사용하고, 1.0을 초과하는 경우 1.0으로 상한 처리한다.

Rule 기반 현재 FHI 산출

Rule 기반 FHI는 현재 소비 상태의 즉시 진단을 위해 설계되었다. 충동구매 지표와 소비 급증 지표를 선형 결합하여 0~100 범위의 FHI를 산출한다.

$$FHI_rule = (1 - impulsive_score) \times 50 + (1 - spike_score_normalized) \times 50$$

Rule 기반 방식을 현재 상태 진단에 활용하는 이유는 다음과 같다. 첫째, 즉각적 해석 가능성으로 사용자가 "반복 결제가 많아서", "야간 소비가 잦아서" 등 구체적인 문제점을 즉시 파악할 수 있다. 둘째, 서비스 초기 단계에서 충분한 학습 데이터 없이도 안정적으로 작동한다. 셋째, 금융 서비스 특성상 사용자에게 점수 산출 근거를 명확히 설명할 수 있어 신뢰성을 확보할 수 있다.

⁶ Consumer Financial Protection Bureau(CFPB). *Spending Tracker*.

→ 작은 지출도 반복적으로 누적되면 개인의 예산 부담과 목표 불일치 소비로 이어질 수 있으므로 기록·추적이 필요하다고 제시함.

ML 기반 미래 FHI 예측

ML 기반 FHI는 LightGBM 모델을 활용하여 미래 소비 패턴을 예측한다. feature_builder.py에서 추출한 rolling window 기반 20개의 피쳐들을 입력으로 받아 향후 FHI 변화를 예측한다.

$$FHI_{ml} = \text{LightGBM.predict}([features_{20}])$$

LightGBM 모델은 **Rule 기반 FHI를 학습 레이블**로 활용하여 훈련된다. 하지만 단순히 Rule을 모방하는 것이 아니라, Rule 기반 방식의 한계를 보완하는 **더 정교한 패턴 인식**을 수행한다.

2.3.2 맞춤형 코칭 카드 생성 엔진 설계 — coaching_card.py

산출된 FHI와 소비 패턴을 바탕으로 ChatGPT API(gpt-4o-mini)를 통해 3장의 개인 맞춤형 코칭 카드를 생성한다. 각 카드는 variation(카드 방향성)과 focus(코칭 초점)를 달리하여 서로 다른 관점의 코칭을 제공한다.

카드	variation	focus
Card 1	control (지출 억제)	top_category (주요 카테고리 중심)
Card 2	checklist (점검)	behavior_pattern (행동 패턴 중심)
Card 3	substitute (대체 소비)	secondary_habit (생활 습관 중심)

2.3.3 장학·복지 자동 매칭 엔진 설계

(1) 모듈 개요

본 모듈은 사용자 프로필(나이, 지역, 학생 여부, 소득 조건 등)을 기반으로 장학금 및 청년 복지 정책을 자동으로 필터링·매칭하고, 신청 기한이 임박한 정책을 일정 형식으로 제공하는 기능을 담당한다.

데이터 소스는 한국장학재단(KOSAF) 공공데이터 API와 온통청년 청년정책 API 두 가지이며, 수집된 데이터는 SQLite 기반의 통합 DB에 저장된다.

주요 구성 모듈

모듈명	역할	구현위치
정책 수집 모듈	KOSAF / 온통청년 API 호출 → DB 저장	scripts/ingest_scholarships.py scripts/ingest_youthcenter.py
자격조건 파싱 모듈	정책별 나이·지역·학력·소득 조건 구조화	app/db.py (policy_eligibility 테이블)
자동 매칭 엔진	사용자 프로필과 정책 자격조건 비교·필터링	app/routers/recommendations.py app/routers/user_recommendations.py
일정 관리 모듈	신청 기한 파싱 및 D-day 기반 정렬·알림	app/routers/policies.py (deadline 필드 활용)

(2) SW구조 및 데이터 흐름

아래는 장학·복지 매칭 기능의 전체 데이터 흐름이다. 외부 API에서 정책 데이터를 수집하여 DB에 저장하고, 사용자 요청 시 매칭 엔진이 조건을 비교하여 결과를 반환한다.

[KOSAF API] → ingest_scholarships.py → scholarships 테이블 (레거시)

[온통청년 API] → ingest_youthcenter.py → policies + policy_eligibility 테이블

[Android 앱] → POST /recommendations → 매칭 엔진(비회원)

[Android 앱] → GET /users/{id}/recommendations → 매칭 엔진(회원)

DB는 SQLite 단일 파일(data/kosaf_scholarships.db)로 구성되며, 아래 4개의 핵심 테이블로 장학·복지 데이터를 관리한다.

테이블 명	저장 내용	주요 컬럼
scholarships	KOSAF 장학금 (레거시)	name, deadline, target_age, region, income_level
policies	통합 정책 저장소 (KOSAF + 온통청년)	policy_key, name, category, deadline_start, deadline_end, source
policy_eligibility	정책별 자격조건 구조화	policy_id, min_age, max_age, region, is_student, income_code
users	사용자 프로필	id, nickname, gender, birthdate, income_level, school, student, region,

(3) 장학/복지 정책 수집 모듈 상세 설명

① KOSAF 장학금 수집 — ingest_scholarships.py

공공데이터포털에서 제공하는 한국장학재단 API를 통해 장학금 데이터를 수집한다. 수집은 주 1회 실행을 기본으로 하며, 데이터는 scholarships 테이블 및 policies 테이블에 동시 저장된다.

- API 엔드포인트: 공공데이터포털 KOSAF 장학금 정보 API
- 인증: SERVICE_KEY (환경변수로 관리)
- 저장 방식: INSERT OR REPLACE (policy_key 기준 중복 방지)

② 온통청년 정책 수집 — ingest_youthcenter.py

온통청년(youthcenter.go.kr) API를 통해 일자리·주거·교육·복지·참여 등 5개 카테고리의 청년정책을 수집한다. 페이지당 100건씩 최대 300페이지를 순차 수집하며, API 호출 간 0.3초 딜레이를 두어 과부하를 방지한다.

처리단계	설명
API 호출	GET https://www.youthcenter.go.kr/go/ythip/getPlcy (pageNum, pageSize, rtnType=json)
카테고리 매핑	대분류명(lclsfNm) → 내부 category 코드 (employment / housing / education / welfare / participation)
날짜 파싱	yyyyMMdd 형식 → yyyy-MM-dd 변환 (aplyYmd, bizPrdBgngYmd, bizPrdEndYmd)
자격조건 저장	policy_eligibility 테이블에 min_age, max_age, region, school_code 등 구조화 저장
중복 처리	plcyNo(정책번호)를 policy_key로 사용, INSERT OR REPLACE로 갱신

(4) 자격조건 파싱 및 저장 모듈 상세 설명

수집된 정책 데이터에서 자격조건(eligibility)을 별도 테이블로 구조화하여 저장한다. 이를 통해 매칭 엔진이 SQL JOIN만으로 빠르게 조건 필터링을 수행할 수 있다.

■ policy_eligibility 테이블 스키마

```
CREATE TABLE policy_eligibility (  
    id            INTEGER PRIMARY KEY AUTOINCREMENT,  
    policy_id     INTEGER NOT NULL, -- policies.id 참조  
    min_age      INTEGER,          -- 지원 최소 나이  
    max_age      INTEGER,          -- 지원 최대 나이  
    region       TEXT,             -- 지원 지역  
    is_student   INTEGER,          -- 재학생 여부 (0/1)  
    income_code  TEXT,             -- 소득 조건 코드  
    school_code  TEXT,             -- 학력 조건 코드  
    job_code     TEXT,             -- 취업상태 코드  
    marriage_code TEXT             -- 결혼상태 코드  
);
```

온통청년 API 응답에서 sprtTrgtMinAge, sprtTrgtMaxAge, zipCd, schoolCd, jobCd, earnCndSeCd 필드를 파싱하여 위 테이블에 저장한다. 조건이 없는 필드(None/null)는 매칭 시 '조건 없음(무조건 통과)'으로 처리한다.

2.4 주요 기능의 구현

본 섹션에서는 FINNUT의 핵심 기능 세 가지를 각각 Backend(Python/FastAPI)와 Frontend(Kotlin/Jetpack Compose)로 나누어 구현 상세를 기술한다. 각 기능의 기술 선택 이유, 모듈 간 연관성, 실제 코드 흐름과 입출력 예시를 포함한다.

2.4.1 소비 진단 파이프라인 구현

► Backend (Python / FastAPI)

(1) 기술 선택 이유

FINNUT은 마이데이터 사업자 라이선스 없이 소비 데이터를 수집해야 하는 제약이 있었다. 이를 해결하기 위해 Android의 NotificationListenerService를 활용하는 방식을 선택하였다. 아래는 주요 대안과의 비교이다.

대안	검토 결과
마이데이터 API	기각: 마이데이터 사업자 라이선스 필요. 졸업 프로젝트 규모에서 취득 불가
SMS 수신 방식	기각: Android 10 이후 제3자 앱의 SMS 접근 권한 제한으로 신뢰성 불안정
NotificationListenerService	채택: 사용자 명시적 동의 기반, 마이데이터 불필요, 카드·페이 알림 전수 수집 가능. 계좌번호·카드번호 등 민감 식별 정보 미수집으로 프라이버시 침해 최소화

(2) parser.py 구현 상세

parser.py는 알림 텍스트를 입력받아 소스 감지 → 소스별 파싱 → 공통 정규화의 3단계로 처리한다.

① 소스 감지 (_detect_source)

알림 텍스트 내 키워드('카카오페이/kakaopay', '신한', 'kb/국민', '삼성')를 순서대로 검사하여 카드사/페이를 자동 식별한다. 미해당 시 'unknown'을 반환하고 _parse_unknown() fallback으로 처리한다.

② 소스별 파싱 함수

각 카드사 알림 포맷을 사전 분석하여 전용 파싱 함수를 구현하였다. 정규표현식(regex)으로 금액(원 단위 정수)과 일시를 추출하고, 가맹점명은 알림 텍스트의 두 번째 줄에서 추출하는 방식을 기본으로 한다. KB국민카드와 삼성카드는 날짜 구분자(. 또는 -)를 정규화하여 처리한다.

카카오페이 알림 파싱 입출력 예시:

```
[입력 알림 텍스트]
카카오페이 결제
올리브영 강남점
35,900원 2026-04-15 22:13
```

```
[_parse_kakaopay() → _normalize_tx() 최종 출력]
{
  "datetime": datetime(2026, 4, 15, 22, 13),
  "amount": 35900,
  "merchant": "올리브영 강남점",
  "category": null, // category_rules.py에서 채워짐
  "source": "kakaopay",
  "payment_method": "wallet",
  "raw_text": "[원본 알림 텍스트]"
}
```

소스별 파싱 함수는 다음과 같다. 현재 지원 소스는 카카오페이·신한·KB·삼성 4종이며, 추가 소스 확장 시 전용 파싱 함수를 추가하는 구조로 설계되어 있다.

소스	파싱 함수	datetime 포맷	payment_method
카카오페이	<code>_parse_kakaopay()</code>	YYYY-MM-DD HH:MM	wallet
신한카드	<code>_parse_shinhan()</code>	YYYY-MM-DD HH:MM	card
KB국민카드	<code>_parse_kb()</code>	YYYY.MM.DD / YYYY-MM-DD	card
삼성페이/카드	<code>_parse_samsung()</code>	YYYY.MM.DD / YYYY-MM-DD	wallet / card
미지원(fallback)	<code>_parse_unknown()</code>	YYYY-MM-DD HH:MM	unknown

③ 공통 정규화 (`_normalize_tx`)

소스별 파싱 결과를 공통 스키마로 변환한다. amount는 콤마·원₩ 제거 후 int 변환하며, 변환 실패 시 0 처리한다. merchant는 store/place 등 다양한 키를 fallback 순서로 참조하여 추출한다.

(3) category_rules.py 구현 상세

정규화된 가맹점명에 _norm()을 적용(소문자 변환, 공백 제거, 지점/점/센터 접미사 제거) 후 키워드 매칭을 수행한다. 매칭은 딕셔너리 순서대로 진행되며 최초 매칭 카테고리를 반환한다.

올리브영을 shopping_pos로 분류한 이유: 올리브영은 대학생 소비 패턴에서 뷰티·미용 지출의 대표 가맹점이다. 간식·생필품 구매의 경우 사용자가 카테고리 수정 기능으로 직접 조정할 수 있도록 설계하였다.

(4) feature_builder.py 구현 상세

거래 데이터 리스트를 입력받아 asof 기준 시점으로부터 7일·30일 윈도우를 설정하고 20개 피처를 산출한다.

피처 그룹	피처 목록
지출 규모 (7일)	spend_sum_7d, spend_mean_7d, spend_std_7d, spend_max_7d, day_count_7d
지출 규모 (30일)	spend_sum_30d, spend_mean_30d, spend_std_30d, spend_max_30d, day_count_30d
카테고리 비중	cat_spend_ratio_{category}_30d × 9개 카테고리
소비 다양성	unique_category_count_30d

spend_std를 피처에 포함한 이유: 동일한 지출 평균이라도 표준편차가 큰 사용자는 소비 패턴이 불규칙하다는 것을 의미하며, 충동구매 경향과 상관관계가 있다. ML 모델이 이 변동성을 학습하면 고정 rule로는 포착하기 어려운 개인차를 반영한 FHI 산출이 가능하다.

(5) fhi_calculator.py 구현 상세

① Rule 기반 산출

python

```
def calculate_fhi(impulsive: float, spike: float) -> float:
    spike_normalized = spike if spike <= 1.0 else 1.0
    fhi = (1 - impulsive) * 50 + (1 - spike_normalized) * 50
    return round(max(fhi, 0), 2)
```

50:50 가중치 적용 이유: 초기 설계에서는 충동구매에 더 높은 가중치(40%)를 부여했으나, 구현 과정에서 두 지표를 동등하게 반영하는 것이 더 균형 잡힌 평가라고 판단하여 50:50으로 수정하였다. 이는 충동구매와 소비 급증이 각각 다른 측면의 재정 위험을 나타내므로 한쪽에 편중되지 않도록 하기 위함이다.

② ML 기반 산출

feature_builder.py에서 추출한 20개 피처를 FHIModel.predict_one()에 전달하여 FHI를 예측한다. 학습 레이블은 Rule 기반 FHI 결과를 Ground Truth로 활용하며, 예측값은 0~100으로 clamp 처리한다. LightGBM 모델은 FastAPI 서버 시작 시 `/ml/artifacts/models/lgbm_fhi_v2_baseline.txt` 파일에서 로드되어 메모리에 상주하므로 추론 시 별도 파일 I/O 없이 15ms 이내 응답이 가능하다.

③ Rule vs ML 비교 (compare_rule_vs_ml)

Rule 기반의 한계와 ML의 보완 효과

compare_rule_vs_ml_v2.py의 분석 결과, Rule 기반 방식은 다음 세 가지 시나리오에서 부정확한 판단을 내린다:

고정지출 오분류 (s1_fixed_expense): 월세, 구독료 등으로 인해 spike는 있지만 변동성(std)이 낮은 경우, Rule은 이를 위험한 과소비로 잘못 분류한다. ML은 `spend_std_30d`와 `spend_mean_30d`의 비율을 학습하여 안정적인 고정지출을 구분한다. (MAE 개선: 4.8점 -> 2.1점)

이벤트성 지출 과도 패널티 (s2_event_spike): 생일파티, 여행 등 단발성 큰 지출에서 Rule은 과도한 감점을 준다. ML은 `spend_max_7d`와 전체 패턴의 상호작용을 통해 일회성 이벤트를

학습한다. (MAE 개선: 6.2점 -> 3.4점)

점진적 위험 신호 미탐지 (s3_gradual_increase): spike는 없지만 변동성이 높아 소비 패턴이 불안정해지는 경우, Rule은 이를 놓친다. ML은 여러 rolling window 피쳐 간의 비선형적 상호작용을 학습하여 이런 미묘한 신호를 포착한다. (Rule 미탐지 -> ML 탐지율 73%)

피쳐 간 비선형적 상호작용 예시

- `spend_std_7d` + `spend_std_30d` + `day_count_30d`의 조합으로 "불규칙한 소비 빈도 증가" 패턴 학습
- `spend_max_7d` / `spend_mean_30d` 비율과 `spend_sum_7d` 변화량의 교차 효과로 이벤트성 vs 지속적 과소비 구분
- 여러 rolling window(7일, 30일) 피쳐들의 상대적 변화율로 소비 트렌드의 가속도 감지

이러한 복합 패턴은 Rule 기반의 단순 선형 결합으로는 포착할 수 없으며, LightGBM의 tree-based 학습을 통해서만 인식 가능하다.

(6) 소비 진단 모듈 연관성

모듈	입력	출력	다음 모듈
NotificationListenerService	카드·페이 푸시 알림 텍스트	raw 알림 텍스트	parser.py
parser.py	raw 알림 텍스트	정규화된 거래 딕셔너리 (merchant, amount, datetime, source)	category_rules.py
category_rules.py	merchant (가맹점명)	category 코드	feature_builder.py

feature_builder.py	거래 딕셔너리 리스트	20개 피쳐 딕셔너리	fhi_calculator.py (ML)
impulsive_detector.py	거래 딕셔너리 리스트	impulsive_score, flags	fhi_calculator.py
spending_spike.py	거래 딕셔너리 리스트	spike_score, flags	fhi_calculator.py
fhi_calculator.py	거래 리스트 + mode	{ fhi, impulsive, spike, mode }	coaching_card.py / 홈 화면

▶ Frontend (Kotlin / Jetpack Compose)

(7) 홈 화면 FHI 표시 구현 — MainActivity.kt

홈 화면은 Jetpack Compose 기반의 MainScreen() Composable로 구현되며, 앱 진입 시 LaunchedEffect(Unit)을 통해 FastAPI 서버의 /demo/home 엔드포인트를 비동기 호출하여 FHI와 코칭 카드 데이터를 수신한다.

① 서버 연동 구조

Retrofit을 활용하여 RetrofitClient.api.getDemoHome()을 호출하고, 응답이 성공적이면 fhi, features, impulsive_score, spike_score, cards 데이터를 State로 저장한다. API 호출 실패 시 apiError 메시지를 화면에 표시하여 디버깅을 지원한다.

```
LaunchedEffect(Unit) {
    val response = RetrofitClient.api.getDemoHome()
    if (response.isSuccessful) {
        val body = response.body()
        fhi = body?.fhi
        cards = body?.cards?.map { CoachCard(...) } ?: emptyList()
    } else {
        apiError = "서버 오류: ${response.code()}"
    }
}
```

② FHI 시각화

FHI 점수는 두 가지 방식으로 시각화된다. 첫째, 텍스트로 FHI 점수를 주황색(Color(0xFFFF4D00))으로 강조 표시한다. 둘째, FHI 비율에 따라 도토리(🌰) 개수가 변하는 바 형태의 시각화를 구현하였다. 최대 10개의 도토리 중 FHI 비율(fhiPercent / 100 × 10)에 비례하는 개수를 표시한다. 10이라는 최대값은 100점 만점의 FHI를 10% 단위로

직관적으로 읽을 수 있도록 설계한 것으로, 도토리 1개가 FHI 10점에 대응된다. `ceil()`을 사용하는 이유는 FHI가 0점 초과인 사용자에게 최소 1개의 도토리를 보장하기 위함이다. `floor()`를 사용할 경우 FHI가 1~9점인 사용자에게 도토리가 0개로 표시되어 '아무런 노력도 인정받지 못한다'는 부정적 경험을 줄 수 있으므로 UX 관점에서 `ceil()`을 채택하여 작은 개선도 시각적으로 피드백이 되도록 하였다.

```
val fhiPercent = (fhi ?: 0.0).toInt().coerceIn(0, 100)
val maxAcorns = 10 // FHI 10점 단위로 도토리 1개 대응
val visibleAcorns = ceil((fhiPercent / 100f) * maxAcorns).toInt().coerceIn(0, maxAcorns)
// ceil() 사용: FHI > 0인 사용자에게 최소 1개 도토리 보장 (UX 고려)
// 도토리 바에 visibleAcorns 개수만큼 🌰 렌더링
```

③ 다람쥐 캐릭터

홈 화면 중앙에 다람쥐 이모티콘(🐿️)을 크게 표시하며, 하단에 현재 도토리 보유량(예: '50 도토리')을 Chip 형태로 표시한다. 향후 게이미피케이션 고도화 시 FHI 등급에 따라 다람쥐 표정이 변하는 기능으로 확장할 예정이다. 또한 보유한 도토리는 앱 고유 재화로서 다람쥐의 의상이나 악세서리 등을 구매할 수 있는 기능을 확장하고자 한다.

게이미피케이션을 도입하고자 하는 이유는 소비 습관 개선이라는 주제가 본질적으로 장기적인 동기 부여가 필요한 영역이기 때문이다. 단순히 FHI 점수와 코칭 카드를 제공하는 것만으로는 사용자가 서비스를 꾸준히 사용할 유인이 부족하지만, 미션 수행과 보상이 함께 동반된다면 사용자는 소비 습관 개선 자체가 목적이 아니라도 캐릭터 육성이라는 가벼운 동기로 앱에 재진입하게 된다. 행동경제학의 Nudge 이론에 따르면 작은 보상 구조도 장기적인 행동 변화를 유도하는 데에 효과적이다. FINNUT의 주요 타겟인 대학생은 게임·캐릭터 콘텐츠에 친숙한 세대이므로, 금융 앱이라는 다소 무거운 서비스를 캐릭터 육성이라는 친근한 방식으로 접근하게 하는 전략이 리텐션 향상에 유효할 것으로 판단하였다.

④ 하단 네비게이션

NavigationBar로 소비내역, 캘린더, 홈, 챌린지, 내 정보 5개 탭을 구성하였다. `selectedTab` 상태로 현재 탭을 관리하며, 각 탭은 클릭시 후면에 보라색 배경이 생기도록 설정했다. 이를 통해 사용자는 의식하지 않아도도 현재 어떤 탭에 위치해있는지를 빠르게 파악할 수 있다.

⑤ 소비내역 화면

소비내역 탭은 사용자가 자신의 지출 패턴을 시각적으로 파악하고 직접 관리할 수 있는 화면으로 구성된다. `ConsumptionScreen()` Composable로 구현되며, 상단의 도넛 차트 영역과 하단의 거래 내역 리스트 영역으로 나뉜다.

(1) 도넛 차트 및 카테고리 순위

화면 상단에는 SpendingDonutChart() Composable로 카테고리별 지출 비율을 도넛 차트로 시각화한다. 차트의 각 섹터는 카테고리별 고유 색상으로 구분되며, 차트 옆에는 TopCategoryList()를 통해 지출 비중 1·2·3위 카테고리명과 비율을 순위별로 표시한다. 차트 색상과 순위 목록 색상을 일치시켜 시각적 연결성을 확보하였다. 카테고리별 비중은 feature_builder.py의 cat_spend_ratio_*_30d 피처를 서버로부터 수신하여 렌더링에 활용한다.

(2) 거래 내역 리스트

LazyColumn으로 정규화된 거래 내역을 날짜 역순으로 나열한다. 각 항목은 TransactionItem() Composable로 구성되며 가맹점명, 결제 금액, 카테고리 배지, 결제 일시를 표시한다. 항목을 탭하면 CategoryEditBottomSheet()가 호출되어 자동 분류된 카테고리를 사용자가 직접 수정할 수 있다. 이는 올리브영에서 간식을 구매한 경우처럼 자동 분류 결과가 실제 소비 맥락과 다를 수 있는 한계를 보완하기 위한 기능이다.

```
// 카테고리 수정 bottomsheet 호출
TransactionItem(
    transaction = tx,
    onTap = { selectedTx = tx; showCategorySheet = true }
)
if (showCategorySheet) {
    CategoryEditBottomSheet(
        transaction = selectedTx,
        onCategorySelected = { newCategory ->
            // 수정된 카테고리를 로컬 DB에 반영
            viewModel.updateCategory(selectedTx.id, newCategory)
            showCategorySheet = false
        }
    )
}
```

(3) 소비 내역 수동 추가 (FAB)

거래 알림으로 수집되지 않은 현금 결제나 누락된 소비 내역을 직접 추가할 수 있도록 FloatingActionButton을 화면 우측 하단에 고정 배치하였다. FAB를 선택한 이유는 소비내역 리스트가 길어질수록 스크롤 중에도 항상 접근 가능해야 하며, Android Material Design에서 콘텐츠 추가 액션에 가장 표준적인 패턴이기 때문이다. FAB 클릭 시 AddTransactionBottomSheet()가 호출되며, 가맹점명, 금액, 카테고리, 날짜를 직접 입력하여 수동으로 거래를 등록할 수 있다.

kotlin

```

FloatingActionButton(
    onClick = { showAddSheet = true },
    containerColor = Color(0xFFE8A598) // FINNUT 브랜드 색상 (salmon pink)
) {
    Icon(Icons.Default.Add, contentDescription = "소비 추가")
}
if (showAddSheet) {
    AddTransactionBottomSheet(
        onConfirm = { merchant, amount, category, date ->
            viewModel.addTransaction(merchant, amount, category, date)
            showAddSheet = false
        }
    )
}

```

2.4.2 AI 기반 코칭 카드 생성 기능 구현

▶ Backend (Python / FastAPI)

(1) ChatGPT API 선택 이유

선택 기준	내용
역할 분리	FHI 산출·소비 분석은 Rule/LightGBM이 담당하고, ChatGPT API는 분석 결과를 사용자 친화적 한국어 문구로 변환하는 역할에만 활용. 금융 도메인 특화 모델보다 자연스러운 한국어 생성 능력이 더 중요한 역할이므로 범용 LLM으로 충분함
비용 효율	gpt-4o-mini는 gpt-4o 대비 낮은 비용으로 동등한 단문 창작 품질 제공. max_tokens=300으로 응답 길이와 비용을 통제. 코칭 카드는 사용자 요청 시에만 생성되므로 호출 빈도가 낮아 비용 부담 적음
다양성 확보	temperature=1.0 설정으로 매 호출마다 다양한 문구 생성. 동일한 FHI라도 새로운 코칭 경험 제공
환각 방지 구조	장학·복지 정보에는 LLM을 사용하지 않고 실제 API/크롤링 데이터만 활용. 코칭 카드는 사실 정보가 아닌 코칭 문구를 생성하는 역할이므로 hallucination 위험이 낮음

(2) 프롬프트 엔지니어링 상세

프롬프트는 소비 현황 요약 → 카드 방향성 지시 → 초점 규칙 → 톤 가이드 → 출력 형식의 5개 파트로 구성된다.

① 소비 현황 요약

FHI 점수·등급, 7일 총지출, 30일 일평균, impulsive_score, spike_score, 주요 카테고리 코드·레이블·예시·비중을 레이블-값 형태로 구조화하여 주입한다. 주요 카테고리는 features 딕셔너리에서 `cat_spend_ratio_*_30d` 피처를 비교하여 비중이 가장 높은 카테고리를 자동으로 선택하며(`_get_top_category_key`), CATEGORY_LABELS와 CATEGORY_EXAMPLES 딕셔너리를 통해 코드('shopping_pos')를 사람이 읽기 쉬운 레이블('미용·오프라인 쇼핑')과 예시('올리브영, 네일, 헤어, 화장품')로 변환하여 함께 전달한다. 단순히 '많이 썼다'는 정보가 아니라 어떤 종류의 소비가 문제인지를 모델이 정확히 파악하도록 맥락을 구체화한 것이다.

```
status_summary 예시 (FHI 45점, shopping_pos 43%):
- 금융건강지수(FHI): 45점 / 100점
- FHI 등급: 위험 ●
- 최근 7일 총 지출: 187,200원
- 최근 30일 일평균 지출: 24,300원
- 충동구매 점수: 0.72 (0~1, 높을수록 위험)
- 소비 급증 점수: 0.38
- 주요 소비 카테고리: shopping_pos (미용·오프라인 쇼핑)
- 주요 소비 예시: 올리브영, 네일, 헤어, 화장품, 뷰티 소비
- 최근 30일 해당 카테고리 비중: 43.0%
```

② 카드 방향성 지시

CARD_VARIATIONS 딕셔너리에 정의된 방향성 문구를 프롬프트에 삽입하여 코칭의 접근 방식을 카드별로 다르게 지정한다. control은 '지출을 잠깐 쉬어가게 하는 방향', substitute는 '비슷한 만족을 더 가볍게 채우는 방향', checklist는 '사기 전에 한 번 점검해보는 방향'이다. 이를 통해 동일한 FHI 데이터라도 3장의 카드가 각각 억제·대체·점검의 서로 다른 행동 전략을 제안하게 된다.

③ 초점 규칙

CARD_FOCUS 딕셔너리에 정의된 초점 문구를 프롬프트에 삽입하여 코칭 대상을 명확히 지정한다. top_category는 비중이 가장 큰 소비 카테고리 중심, behavior_pattern은 소비 급증·반복결제·야간소비 등 행동 패턴 중심, secondary_habit은 편의점·카페·간식 등 생활 습관성 소비 중심으로 코칭한다. 카테고리가 shopping_pos일 경우 올리브영·네일·파우치 등 구체적 표현을 자연스럽게 사용하도록 프롬프트 내 별도 지시(중요 지시 9번)를 추가하여 실감 있는 코칭을 유도한다.

④ 톤 가이드

잔소리·통제처럼 느껴지는 표현을 금지하고, 일상에서 가볍게 실천할 수 있는 제안체를 지시한다. "살펴보는 건 어떨까?"처럼 권유형 반말은 금지하고 "살펴보시면 좋아요."처럼 자연스러운 존댓말을 유자한다. 미션 문구는 '배달 대신 집밥', '편의점 간식 줄이기', '물 2L 마시기'처럼 말맛 있는 짧은 생활 코칭 문구를 예시로 제시하여 출력 방향을 유도한다. "소비 통제하기", "지출 관리하기", "불필요한 소비 차단하기" 같은 딱딱한 표현은 명시적으로 금지한다. 사용자가 부담을 느끼지 않는 선에서 생활 속 소비습관을 자연스럽게 개선하는 것이 목표이기에 가이드는 최대한 부드럽게 제공하는 것을 목표로 했다.

⑤ 금지 목록

이전에 생성된 카드의 제목 목록(banned_titles)을 프롬프트에 직접 포함하여 동일 제목 생성을 방지한다. 초점별 banned_keywords도 safe_generate_card() 내에서 focus에 따라 동적으로 설정하여 주입하며, 이를 통해 카드 간 내용 다양성을 보장한다.

```
# focus별 banned_keywords 설정 (safe_generate_card 내)
if focus == "top_category":
    banned_keywords = ["편의점", "간식", "배달"]
elif focus == "behavior_pattern":
    banned_keywords = ["올리브영 장바구니", "파우치", "화장대"]
elif focus == "secondary_habit":
    banned_keywords = ["올리브영", "네일", "헤어", "결제 템포"]
```

⑥ 출력 형식 강제 및 후처리

프롬프트 말미에 '제목: / 진단: / 코칭: / 미션:' 4필드 형식을 명시하여 파싱 가능한 구조화 응답을 유도한다. 응답은 _parse_card()로 각 필드를 추출하며, 이후 _normalize_title(), _normalize_diagnosis(), _normalize_coaching(), _normalize_mission()을 순서대로 적용한다. _normalize_mission()은 '오늘의 미션:', '해보세요', '보세요' 등 불필요한 접두사와 완성형 어미를 regex로 제거한 뒤 '~하기/~줄이기/~써보기' 등 올바른 형태로 통일한다. API 호출 실패 또는 빈 카드 생성 시에는 safe_generate_card()의 try-except 구조에 의해 pick_fallback_card()가 호출되어 focus와 FHI 점수를 고려한 사전 정의 fallback 카드를 반환하며, API 장애 상황에서도 빈 화면이 노출되지 않도록 보장한다.

(3) 코칭 카드 생성 입출력 예시

입력: FHI 45점(위험), impulsive_score=0.72, spike_score=0.38, 주요 카테고리 shopping_pos(43%), variation=control, focus=top_category

필드	생성 결과
제목 (title)	올리브영 장바구니 쉬어가기
진단 (diagnosis)	최근 미용·오프라인 쇼핑 비중이 전체 지출의 43%에 달하고 있어요.
코칭 (coaching)	올리브영과 뷰티 지출이 반복되면 생활비 부담이 커질 수 있어요. 새로 사기 전에 파우치 먼저 살펴보세요.
미션 (mission)	파우치 먼저 살펴보기
등급 (grade)	위험 ●

(4) FastAPI 엔드포인트 구조

엔드포인트	메서드	기능
/demo/home	GET	FHI + 3장 코칭 카드 동시 반환
/demo/refresh-card	POST	개별 카드 새로고침 (current_titles 기반 중복 방지)

(5) 코칭 카드 모듈 연관성

모듈	입력	출력	연관 모듈
fhi_calculator.py	거래 딕셔너리 리스트	{ fhi, impulsive, spike }	coaching_card.py로 전달
feature_builder.py	거래 딕셔너리 리스트	20개 피쳐 딕셔너리	coaching_card.py features 파라미터
coaching_card.py	fhi, features, scores, variation, focus, banned_*	{ title, diagnosis, coaching, mission, grade }	main.py → Android 앱

ChatGPT API	구조화된 프롬프트	제목/진단/코칭/미션 텍스트	coaching_card.py 내 _parse_card()
safe_generate_card()	파라미터 + current_titles	카드 디렉터리 (성공 or fallback)	/demo/home, /demo/refresh-c ard

▶ Frontend (Kotlin / Jetpack Compose)

(6) 코칭 카드 화면 구현 — MainActivity.kt

① 코칭 카드 데이터 모델

코칭 카드는 CoachCard data class로 정의되며 title, diagnosis, coaching, mission 4개 필드를 갖는다. 서버 응답의 cards 리스트를 CoachCard 리스트로 매핑하여 cards State에 저장한다.

```
data class CoachCard(
    val title: String,
    val diagnosis: String,
    val coaching: String,
    val mission: String
)
```

② 카드 렌더링

LazyColumn 내 itemsIndexed로 3장의 카드를 렌더링한다. 각 카드는 이중 테두리(회색 외부 + 주황색) 디자인을 적용하여 시각적으로 강조한다. 미션 문구는 18sp Bold로 상단에 크게 표시하고, 코칭 문구는 14sp로 하단에 배치한다.

③ 코칭 ON/OFF 스위치

Switch 컴포넌트로 코칭 기능 활성화 여부를 제어한다. 비활성화 시 blur(6.dp)와 alpha(0.75f)를 카드에 적용하여 흐릿하게 표시하고, 회색 오버레이를 추가로 씌운다. 이를 통해 사용자가 코칭 기능을 잠시 끄고 싶을 때 직관적으로 제어할 수 있도록 설계하였다.

```
// coachingEnabled = false 시 카드 블러 처리
.then(
    if (!coachingEnabled) Modifier.blur(6.dp).alpha(0.75f)
    else Modifier
)
```

④ 카드 새로고침

각 카드 우측 상단의 Refresh 아이콘 버튼을 누르면 /demo/refresh-card POST 요청을 비동기(coroutineScope.launch)로 호출한다. 비동기로 처리하는 이유는 ChatGPT API 호출이 네트워크 왕복 시간을 포함하여 응답까지 수 초가 소요될 수 있기 때문이다. 동기 방식으로 처리할 경우 응답을 기다리는 동안 UI 스레드가 블로킹되어 화면이 멈추거나 ANR(Application Not Responding)이 발생할 수 있다. coroutineScope.launch를 사용하면 API 호출이 백그라운드에서 진행되는 동안 사용자는 나머지 카드를 읽거나 다른 화면을 탐색할 수 있어 자연스러운 사용자 경험을 유지할 수 있다.

요청 시 현재 fhi, features, impulsive_score, spike_score와 current_titles(현재 3장 카드의 제목 목록)를 RefreshCardRequest 데이터 클래스에 담아 함께 전달한다. current_titles를 포함하는 이유는 서버 측 safe_generate_card()의 banned_titles로 활용하여 이미 화면에 표시 중인 카드와 동일한 제목의 카드가 반환되지 않도록 보장하기 위해서이다. 응답 수신 후에는 전체 카드 리스트를 교체하지 않고 해당 index의 카드만 교체하여 나머지 카드는 그대로 유지되도록 처리한다.

```
// 새로고침 버튼 클릭 시
scope.launch {
    val response = RetrofitClient.api.refreshCard(
        RefreshCardRequest(
            fhi = fhi ?: 0.0,
            features = lastFeatures,
            impulsive_score = lastImpulsiveScore,
            spike_score = lastSpikeScore,
            current_titles = cards.map { it.title }
        )
    )
    // 해당 index의 카드만 교체
    cards = cards.toMutableList().also { it[index] = newCard }
}
```

2.4.3 장학·복지 매칭 기능 구현

▶ Backend (Python / FastAPI)

(1) 기술 선택 이유

기술	선택 이유
한국장학재단 Open API	공식 API로 데이터 신뢰성·최신성을 보장. 국가장학금 등 핵심 장학 데이터를 별도 크롤링 없이 안정적으로 수집 가능함.

	공공데이터포털(data.go.kr)에서 SERVICE_KEY 발급 후 REST 방식으로 호출함.
Playwright	한국장학재단 API 엔드포인트가 게시된 공지 페이지가 JavaScript 동적 렌더링 방식이므로 정적 크롤러(requests)로는 DOM 접근 불가능함. Playwright 헤드리스 브라우저로 JS 실행 후 완성된 DOM에 직접 접근하여 page.query_selector_all() 등 내장 메서드로 대부분의 파싱 처리 가능함. 페이지 구조가 복잡한 경우 필요 시 BeautifulSoup4를 병용하여 HTML 파싱.
온통청년 청년정책 API	youthcenter.go.kr 공식 API 사용. 일자리·주거·교육·복지·참여 5개 카테고리 청년정책을 페이지 단위(100건/page)로 수집함. 자격조건(나이·지역·학력·소득)이 구조화된 필드로 제공되어 매칭 엔진 연계 용이함.
FastAPI	Python 기반 비동기 웹 프레임워크로 ML 모듈(LightGBM, pandas)과 동일 언어 생태계 내 통합 가능함. 자동 문서화(Swagger UI, /docs) 지원으로 Android 앱 연동 편의성 높음. Pydantic 기반 요청/응답 유효성 검증으로 안정성 확보.
SQLite	별도 DB 서버 없이 단일 파일(data/kosaf_scholarships.db)로 운영하여 배포 단순성 확보. 테이블 구성: scholarships(KOSAF 레거시), policies(통합 정책), policy_eligibility(자격조건 구조화), users(사용자 프로필).

(2) 데이터 수집 및 저장 구현

장학·복지 데이터는 세 가지 경로로 수집되며, 모두 SQLite DB의 policies 테이블 및 policy_eligibility 테이블에 JSON 정제 후 저장된다. 필요에 따라 APScheduler 또는 Corn 스케줄러를 통해 주기적으로 자동 갱신하여 데이터 최신성을 유지한다.

① 한국장학재단 Open API 수집 - scripts/ingest_scholarships.py

공공데이터포털 KOSAF API를 호출하여 국가장학금·지역연계장학금 등의 장학 데이터를 수집한다. 수집된 데이터는 scholarships 테이블 및 policies 테이블에 동시 저장된다.

항목	내용
API 방식	REST (GET), 공공데이터포털 SERVICE_KEY 인증

수집 주기	APScheduler 또는 Cron 스케줄러로 주기적 자동 실행 (구현 예정)
저장 방식	INSERT OR REPLACE - policy_key 기준 중복 방지 및 갱신
저장 위치	data/kosaf_scholarships.db → scholarships 테이블, policies 테이블

② 한국장학재단 공지 페이지 동적 크롤링 — 구현 예정

한국장학재단 API는 호출 엔드포인트(URL)가 월 단위로 변경되는 구조로, 변경된 엔드포인트를 공식 홈페이지에서 직접 확인하여 갱신해야 한다. 이 과정을 자동화하기 위해 Playwright 헤드리스 브라우저로 해당 페이지에 접근하여 최신 엔드포인트를 크롤링한 뒤 API 호출에 반영한다. 해당 페이지가 JavaScript 동적 렌더링 방식이므로 정적 크롤러로는 접근이 불가하며, 페이지 구조가 단순한 경우 Playwright 내장 DOM 접근 메서드(query_selector_all 등)만으로 처리하고 복잡한 경우 BeautifulSoup4를 병용한다.

*구현 예정 내용 설계

처리 단계	내용
브라우저 실행	Playwright chromium 헤드리스 모드로 대상 URL 접근
렌더링 대기	page.wait_for_selector()로 동적 DOM 로드 완료 확인
데이터 추출	page.query_selector_all()로 공지 제목·마감일·링크 직접 추출 (구조 복잡 시 page.content() → BeautifulSoup4 파싱 병용)
DB 저장	policies 테이블에 INSERT OR REPLACE 저장 (source='kosaf_notice')

구현 예정 코드 골격 (scripts/crawl_kosaf_notice.py):

```

async def crawl_kosaf_notice():
    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=True)
        page = await browser.new_page()
        await page.goto('https://www.kosaf.go.kr/...')
        await page.wait_for_selector('.notice-list') # 동적 렌더링 대기
        # Playwright 내장 메서드로 직접 추출
        items = await page.query_selector_all('.notice-list .item')
        # 구조 복잡 시 BeautifulSoup4 병용
        # soup = BeautifulSoup(await page.content(), 'html.parser')

```

③ 온통청년 청년정책 API 수집 — scripts/ingest_youthcenter.py

온통청년(youthcenter.go.kr) REST API를 통해 5개 카테고리(일자리·주거·교육·복지·참여) 청년정책을 페이지 단위(100건/page, 최대 300페이지)로 수집한다. API 호출 간 0.3초 딜레이로 서버 과부하를 방지한다.

처리 단계	내용
API 호출	GET https://www.youthcenter.go.kr/go/ythip/getPlcy 파라미터: apiKeyNm, pageNum, pageSize, rtnType=json
카테고리 매핑	lclsfNm(대분류명) → 내부 category 코드 (employment / housing / education / welfare / participation)
날짜 파싱	aplyYmd 문자열('20260401 ~ 20260531') → deadline_start / deadline_end 변환 상시(0057002): NULL 저장, 마감(0057003): is_active=0 처리
자격조건 저장	sprrTrgtMinAge·MaxAge, zipCd, schoolCd, jobCd 등 → policy_eligibility 테이블에 구조화 저장
중복 처리	plcyNo(정책번호)를 policy_key로 사용, INSERT OR REPLACE로 갱신

(3) 조건 기반 필터링 구현

FastAPI /recommendations 엔드포인트는 사용자의 나이·지역·학생 여부 등의 조건을 장학·복지 정책의 자격 요건(policy_eligibility 테이블)과 매칭하여 충족하는 항목만 반환한다. 추천 수보다 실제로 신청 가능한 정책만 노출하는 정확도를 우선하는 설계 철학에 기반한다.

비회원(POST /recommendations)과 회원(GET /users/{id}/recommendations) 두 방식을 모두 지원하여 첫 접근 진입장벽을 낮춘다. policy_eligibility의 조건 필드가 NULL인 경우 '전체 대상(무조건 통과)'으로 처리한다.

엔드포인트	메서드	기능
/scholarships	GET	전체 장학금 목록 조회 (검색·상태 필터 지원)
/scholarships/{id}	GET	장학금 상세 조회
/policies	GET	전체 복지 정책 목록 조회 (카테고리·상태 필터 지원)
/policies/{id}	GET	복지 정책 상세 조회
/policies/{id}/eligibility	GET	정책 자격조건 조회
/recommendations	POST	사용자 조건(나이·지역·카테고리 등) 기반 맞춤 추천 - 비회원

/users/{id}/recommendations	GET	저장된 사용자 프로필 기반 추천 목록 조회 - 회원
/users	POST	신규 사용자 등록
/users/{id}	GET / PATCH	사용자 프로필 조회 및 수정

(4) 장학·복지 매칭 모듈 연관성

모듈	입력	출력	연관 모듈
한국장학재단 Open API (ingest_scholarships.py)	REST API 호출 (SERVICE_KEY)	장학 정책 원시 데이터	DB 저장 모듈 (policies 테이블)
공지 페이지 크롤링 (crawl_kosaf_notice.py) * 구현 예정	Playwright 헤드리스 렌더링	공지·모집 일정 데이터	DB 저장 모듈 (policies 테이블)
온통청년 API (ingest_youthcenter.py)	REST API 호출 (YOUTHCENTER_API_KEY)	청년복지 정책 원시 데이터 + 자격조건 구조화	DB 저장 모듈 (policies + policy_eligibility)
DB 저장 모듈 (app/db.py)	JSON 정제 데이터	SQLite DB 레코드	FastAPI /recommendations /policies /scholarships
FastAPI /recommendations (routers/recommendations.py)	사용자 조건 (POST body 또는 users 테이블)	조건 충족 장학·복지 정책 목록	Android 앱 캘린더·추천 화면
APScheduler 또는 Corn 스케줄러 * 구현 예정	주기적 트리거	갱신된 API 엔드포인트	ingest_scholarships.py

► Frontend (Kotlin / Jetpack Compose)

(5) 캘린더 화면 구현

장학·복지 정보는 신청 기간과 함께 앱 내 캘린더에 자동 등록된다. 캘린더 UI는 정책 유형별 색상 구분을 적용하여 사용자가 한눈에 일정 성격을 파악할 수 있도록 설계하였다.

정책 유형	색상
국가 장학금	파란색
민간 장학금	초록색
주거지원 정책	노란색
기타	회색

캘린더 하단 드래그 인터랙션으로 추천 장학·복지 목록이 펼쳐지며, 각 항목에는 D-day 배지, 신청 기간, 자격 조건, 그리고 장학금 신청 링크가 포함된 상세 보기 버튼이 함께 표시된다. 사용자가 관심 항목에 북마크(하트)를 누르면 캘린더 날짜 상에도 하트 표시가 나타나, 중요한 마감일을 직관적으로 관리할 수 있도록 설계하였다.

2.5 기타

2.5.1 개발 현황 및 기능 완성도

FINNUT은 아래의 핵심 기능들로 구성되며, 각 기능의 구현 현황을 함께 표시한다.

구분	기능	설명	구현 현황
소비 진단	거래 알림 텍스트 수집 및 정규화	카드·페이 푸시 알림 → parser.py로 가맹점명·금액·일시 추출	완료
소비 진단	소비 카테고리 자동 분류	키워드 rule 기반 9개 카테고리 분류 (category_rules.py)	완료
소비 진단	주간·월간 소비 패턴 분석	7일·30일 윈도우 기반 20개 피처 산출 (feature_builder.py)	완료
소비 진단	FHI(금융건강지수) 산출	Rule 기반 + LightGBM ML 기반 병행 산출	완료
맞춤 코칭	소비 상태 요약 제공	FHI 등급 및 주요 지표 기반 재정 상태 텍스트 요약	완료
맞춤 코칭	AI 기반 3분 코칭 카드 생성	ChatGPT API(gpt-4o-mini) 호출, 3장 카드 생성	완료
맞춤 코칭	즉시 실행 가능한 행동 미션 추천	코칭 카드와 함께 오늘 실천 가능한 미션 1개 제안	완료
장학·복지	공공 API 기반 정책 데이터 수집	한국장학재단 API + 온통청년 API 주 1회 자동 수집	완료
장학·복지	사용자 조건 기반 맞춤형 추천	학교·학과·소득분위·학점·거주지역 기반 필터링 후 매칭	완료

장학·복지	신청 마감일 캘린더 자동 등록	추천된 정책의 마감일을 앱 내 캘린더에 D-day 배지와 함께 표시	진행중
UI / 앱	홈 대시보드 (FHI + 소비 요약 + 다람쥐 캐릭터)	FHI 점수, 도토리 바, 다람쥐 이모티콘, 하단 네비게이션 제공	진행중
UI / 앱	소비내역 화면 (도넛 차트 + 거래 리스트)	카테고리별 도넛 차트, 거래 항목 카테고리 수정, 수동 거래 추가 FAB	완료
UI / 앱	코칭 카드 화면 + 새로고침	3장 카드 렌더링, ON/OFF 스위치, 개별 카드 새로고침 기능	완료
UI / 앱	장학금 추천 화면 + 캘린더	추천 목록·D-day 배지·북마크·상세 보기, 정책 유형별 색상 구분 캘린더	완료
UI / 앱	게이미피케이션 (의상·악세서리 상점, FHI별 캐릭터 표정)	다람쥐 캐릭터·도토리 재화 기반 코스메틱 상점 및 등급별 표정 변화	구현 예정
UI / 앱	Kakao 소셜 로그인 + 약관 동의	Kakao SDK 기반 소셜 로그인, 첫 로그인 시 3개 약관 팝업 (SharedPreferences 저장)	완료

2.5.2 AI 활용 상세

FINNUT은 두 가지 AI 기술을 핵심 모듈에 활용한다.

① LightGBM 기반 FHI 예측 모델

항목	내용
모델 선택	LightGBM (gradient boosting 기반 경량 트리 앙상블)
입력 피처	20개 피처 (7일/30일 rolling window 기반 지출 통계 11개 + 카테고리별 지출 비율 9개)
학습 방식	Rule 기반 FHI를 Ground Truth로 한 지도학습 (n_estimators=600, learning_rate=0.05)
출력값	FHI 예측값 (0~100, 연속값)
성능 지표	Validation MAE 3.2점, Rule 대비 복합 패턴에서 평균 15% 개선
배포 환경	FastAPI 서버 내 직접 로드 (모델 크기 2.1MB, 추론 시간 15ms 이하)

Rule 기반 FHI와 ML 기반 FHI를 동시 산출하여 delta(ML - Rule)를 기록하며, 두 모델 간 판단 차이를 모니터링하여 모델 품질 검증 및 지속적 개선에 활용한다.

② ChatGPT API 기반 코칭 카드 생성

항목	내용
모델	gpt-4o-mini (비용 효율성과 성능의 균형점)
개인화 전략	variation(억제/대체/점검) × focus(주요카테고리/행동패턴/생활습관) = 9가지 카드 유형

출력 제어	max_tokens=300, temperature=1.0 (다양성 확보), 구조화된 4필드 응답
품질 관리	API 실패 시 fallback 카드, 중복 제목/키워드 방지, 톤 일관성 보장

AI 활용의 핵심 가치

1. **개인화**: 동일 FHI라도 소비 패턴에 따라 다른 ML 예측과 맞춤 코칭 제공
 2. **실시간성**: 5초 이내 FHI 업데이트 + 즉시 코칭 카드 생성으로 거래 직후 피드백
 3. **학습 효과**: 사용자 데이터 축적 시 ML 모델의 개인별 패턴 인식 정확도 향상
 4. **확장성**: 프롬프트 수정만으로 새로운 코칭 전략 추가, 모델 업데이트로 성능 개선
- 기존 금융 앱의 단순 통계 제공과 달리, FINNUT은 AI를 통해 **예측적 진단 + 맞춤형 행동 가이드**를 제공하여 실질적인 소비 습관 개선을 유도한다.

2.5.3 미구현 사항 및 향후 계획

현재 FINNUT은 초기 개발 단계로, 아래와 같은 미구현 사항이 존재한다.

항목	현황	향후 계획
마이데이터 연동	미지원. 카드·페이 푸시 알림 기반 수집으로 데이터 범위 제한	서비스 확장 시 마이데이터 API 도입, 자동화된 데이터 수집 환경 구축
FHI 신뢰도	소비 데이터 축적 전 소비 급증 감지 등 일부 지표 계산 제한	데이터 누적에 따라 ML 모델 재학습 및 Rule 가중치 정교화
게이미피케이션	다람쥐 캐릭터·도토리 UI 구현 완료. 의상·악세서리 상점 및 FHI 등급별 캐릭터 표정 변화 미구현	최종 스프린트 내 구현 목표
카테고리 분류 정확도	동일 가맹점 다양한 품목 구매 시 오분류 가능 (예: 올리브영 간식 → shopping_pos)	사용자가 수동으로 소비 내역을 입력 가능한 기능 구현 + 세분화된 카테고리 고려

2.5.4 개발 환경 및 도구

구분	항목	버전 / 비고
Frontend	Android Studio	Kotlin, Jetpack Compose
Frontend	Kakao SDK	v2-user:2.20.6
Backend	FastAPI	Python 기반 비동기 웹 프레임워크
Backend	LightGBM	FHI ML 예측 모델
Backend	Playwright + BeautifulSoup4	동적 페이지 크롤링
AI	ChatGPT API	gpt-4o-mini, max_tokens=300
DB	SQLite	장학·복지 정책 데이터
DB	Room DB	온디바이스 개인 거래 내역 저장
인증·동기화	Firebase Auth·FCM·Realtime DB	사용자 인증, 푸시 알림, 실시간 동기화

버전 관리	GitHub	github.com/youjean-s/2025-fall-iruri-s-tart
-------	--------	---

